Ordered - Key Storage Solution

ghostwydwu（武勇栋）

Table of Contents

- Applicable Scenarios
- Interfaces
- Architecture
- Metadata
- Replica Synchronization
- Storage Engine
- Resource Scheduling
- Fault Detection & Recovery

Applicable Scenarios

Tree-structured data — index directories
Key: QQ + parent directory ID + directory/file ID

✓One-dimensional time series — Feed streams
Key: QQ + timestamp

✓Two-dimensional spatial ordering — LBS-enabled applications
Key: GeoHash(longitude, latitude)

●- Support single-record insertion and batch range scanning.

●- Businesses do not need pre-batching records, eliminating traffic amplification issues.

●- Reduce I/O overhead for batch range fetching and optimize performanc

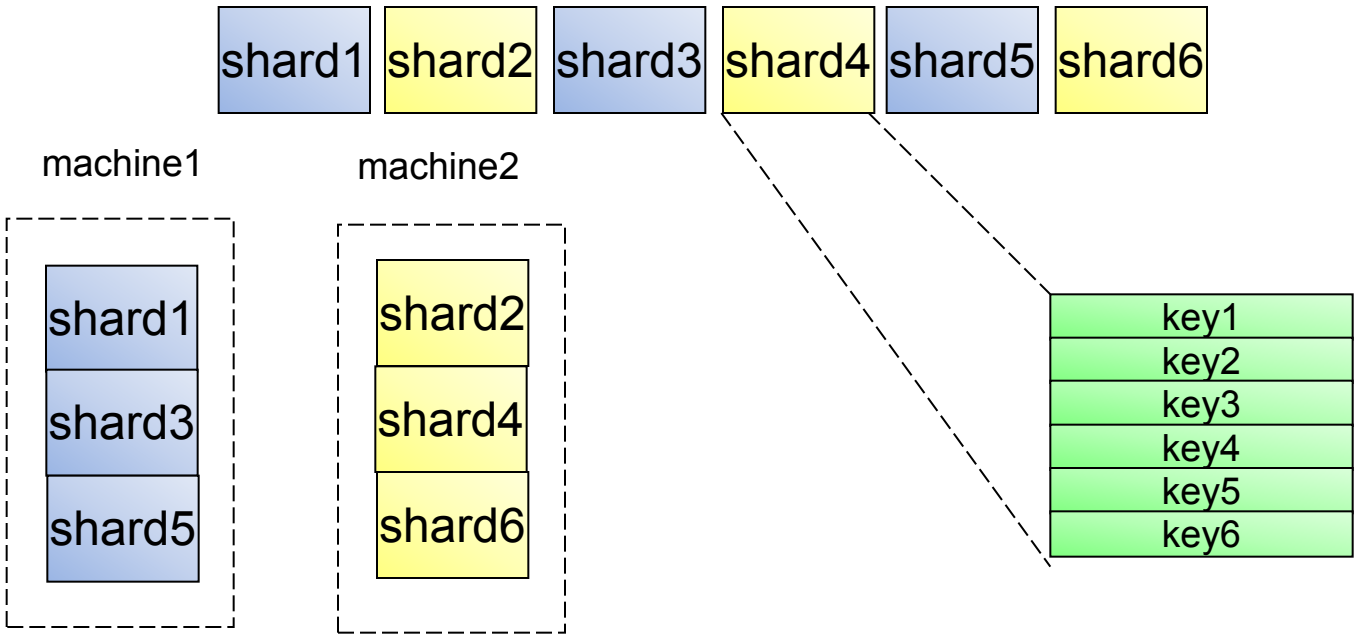
Interface

Single-record APIs
- Get(Key, Value)
- Set(Key, Value)
- Del(Key)

Batch ordered-key APIs
- GetBatch(Begin_key, End_key, Value_list)
- SetBatch(Key_value_list)
- DelBatch(Begin_key, End_key)

Key-Value Length Limits
- Key: Recommended ≤ 1 KB; maximum supported 10 KB
- Value: Maximum supported 512 KB
- Maximum payload for batch fetch APIs: 2 MB

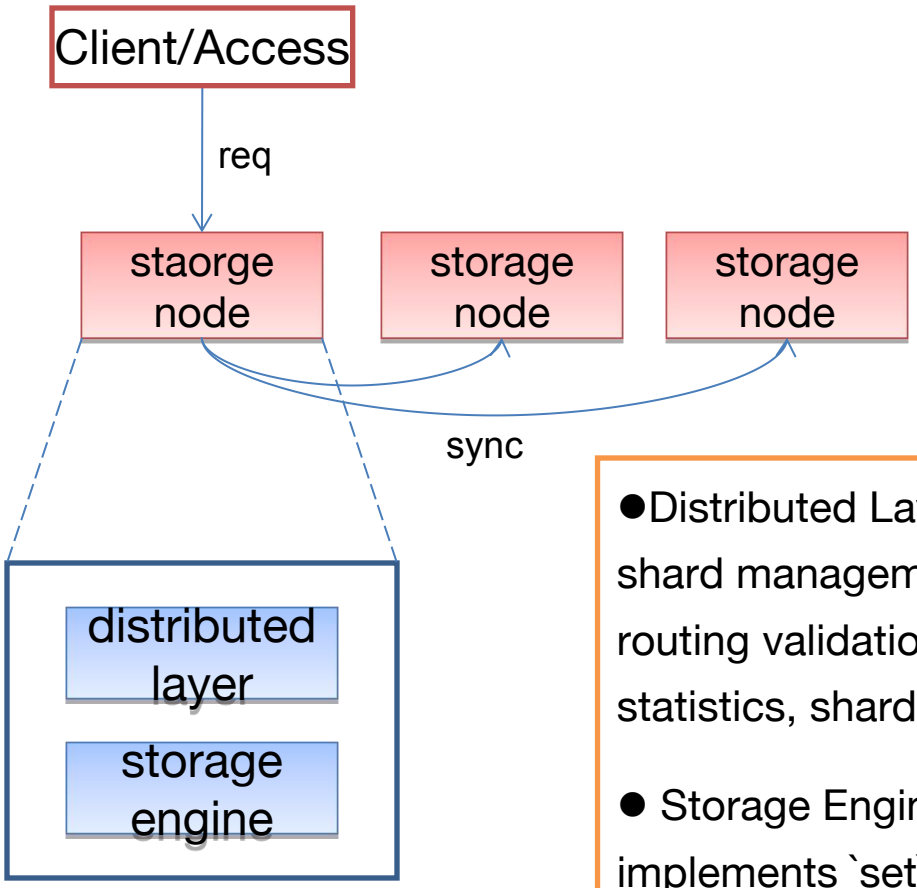
Basic Idea



Core Challenges

1. Given a key range request: how to locate the corresponding shard, host and disk (**routing**).
2. Within each shard: how to store keys in sorted order (**storage engine**).

Architecture – Module Structure



resource manage

resource orchestrator

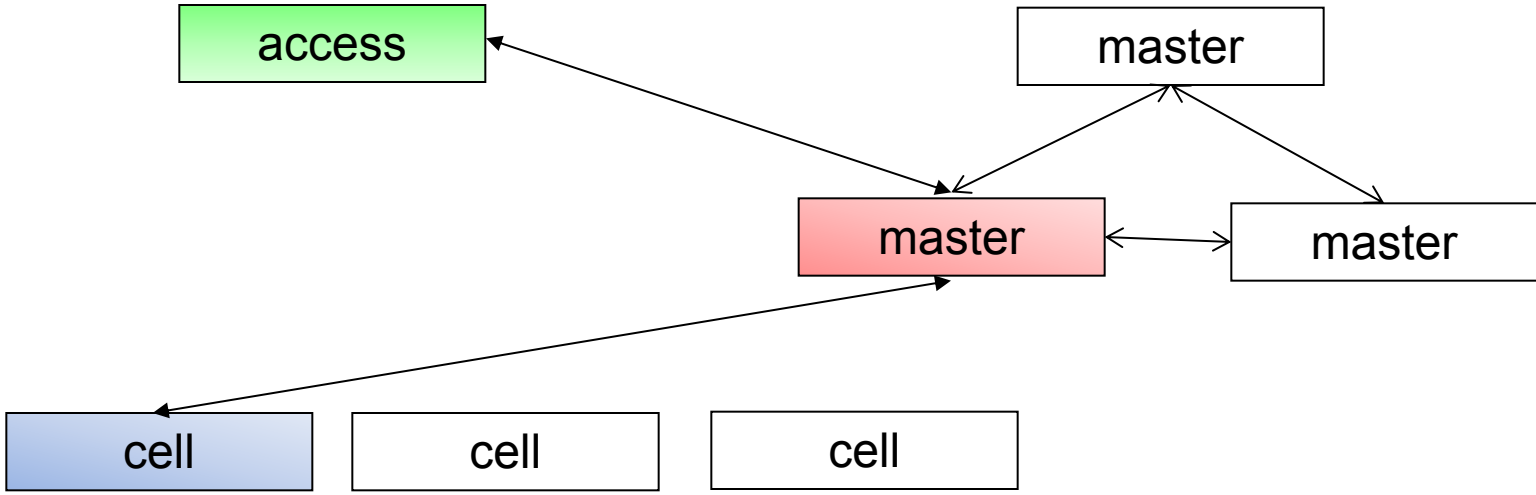
- Distributed Layer: Handles distributed logic: shard management, request processing, routing validation, replica synchronization, statistics, shard migration, etc.
- Storage Engine: Pure local storage layer; implements `set` / `get` / `del` and corresponding batch primitives. No distributed logic.

Meta Data

- Routing Metadata
- - Logical partitioning: range - based logical sharding.
- - Storage: Shard metadata and chained - routing entries for migration are kept in shared memory, indexed via tree structures for fast lookup.
- - High - availability: 3 master nodes run the Raft consensus protocol for metadata fault - tolerance.

Routing Architecture — Sharding

- Logical sharding by key range.
- - Metadata HA: Raft protocol across three master nodes.
- - Primary cell maintains a local shard table to validate whether incoming request ranges match local shard boundaries.

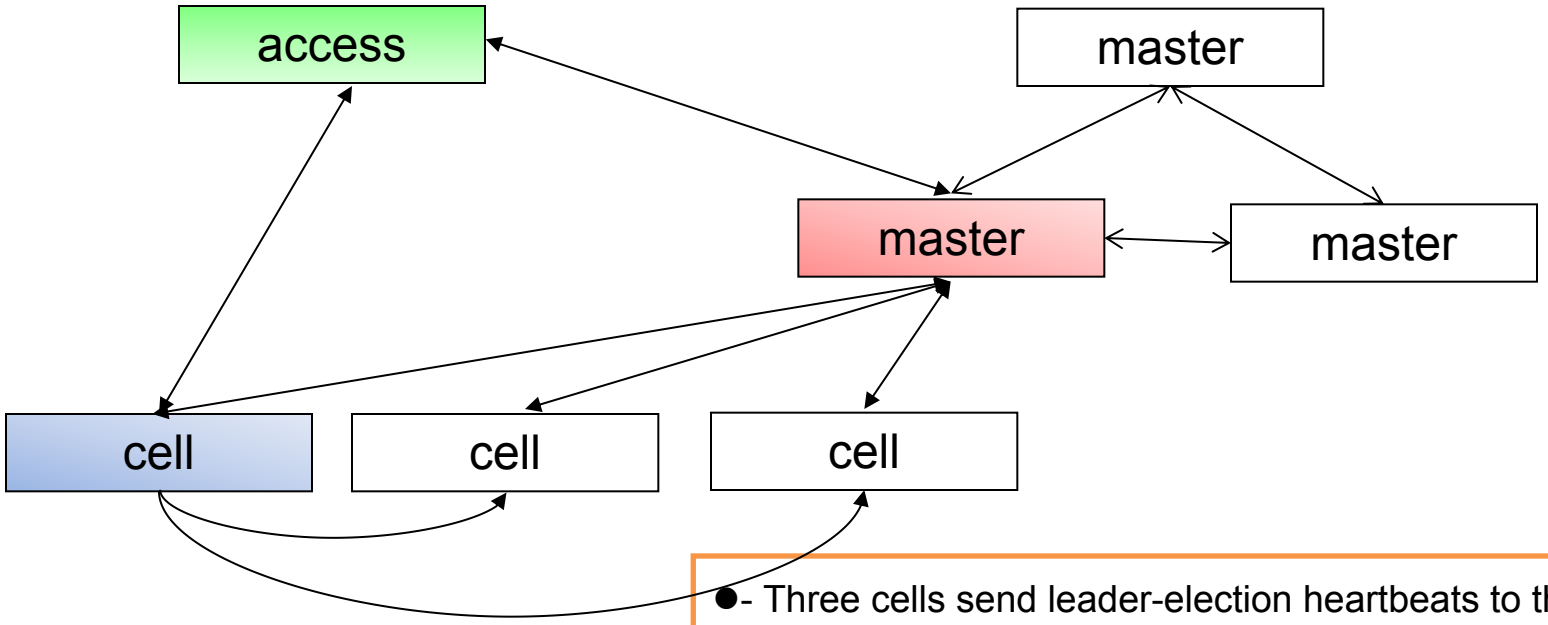


sharded table field

shard id	seq	keylen	keymax	master	disk	follower1	state	follower2	state	valite bit	reservee

Routing Architecture —Primary - Replica Failover

●storage cells periodically sync primary-cell IP information from masters.



●- Three cells send leader-election heartbeats to the master every Ts.

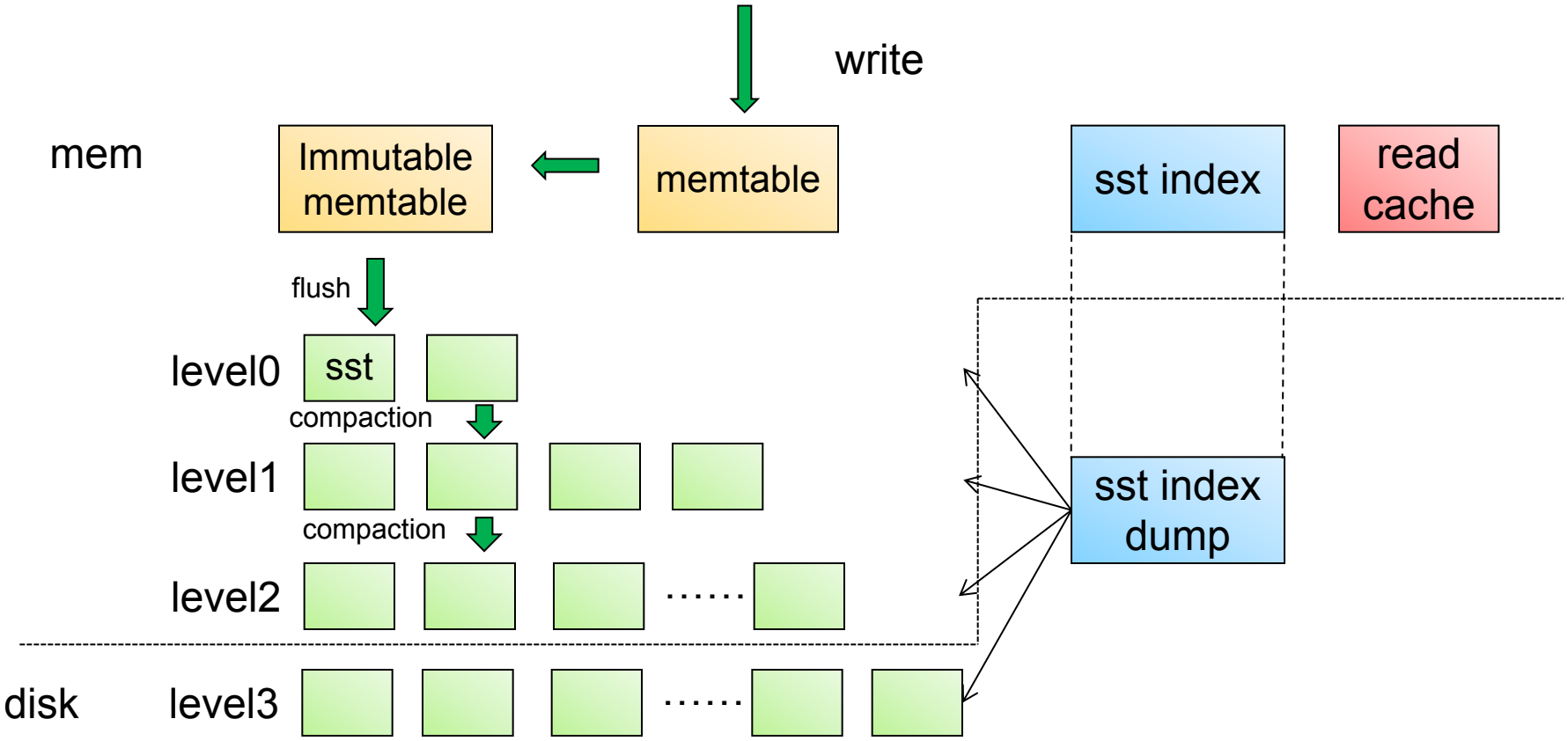
●- The primary cell carries its item number during synchronization. If it detects a higher item number on another cell, it returns failure and demotes itself.

●- Only the primary cell serves Access requests; replica cells deny all Access traffic.

Replica Synchronization & Primary Switchover

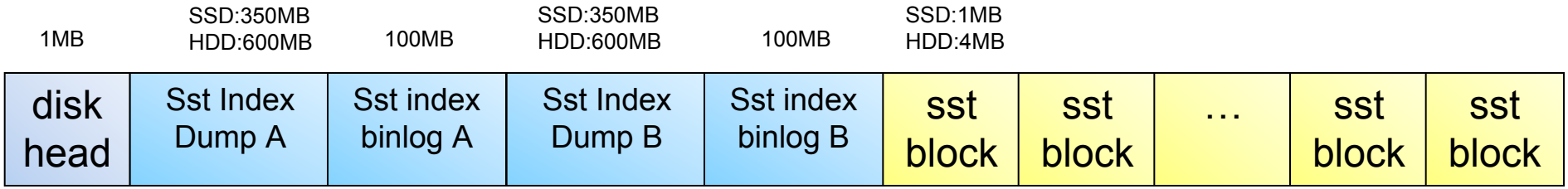
- Placement: fixed – group pairing or global grouping.
- Synchronization granularity: per storage – node level.
- Consistency semantics: `all – replica` acknowledgment, controlled via version numbers.

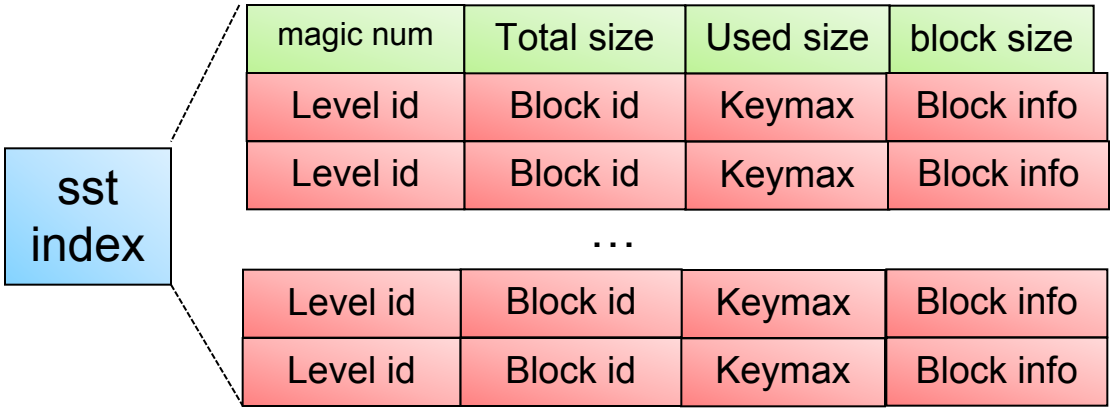
Storage Engine — Architecture



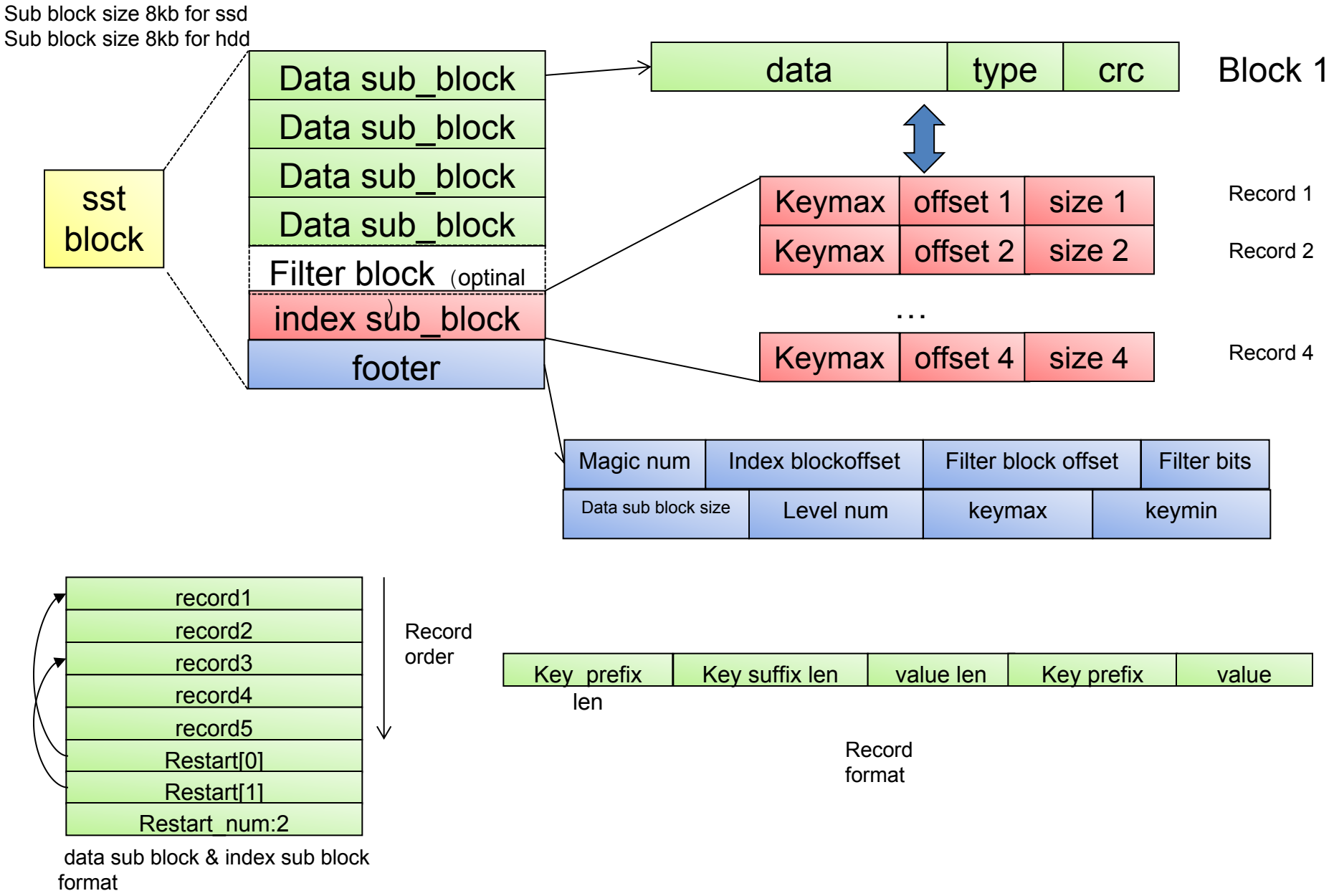
sst block： physically stores ordered key-value records.
sst index： in-memory index of on-disk SST files.
Memtable： active write buffer; KV entries kept sorted by skip-list.
Immutable memtable： sealed memtable after filling; queued for flush to disk
Read cache

Storage Engine — Disk Layout





Storage Engine — Disk Layout



Storage Engine — Disk Layout

Bloom filter（optional）
One bloom filter per SST block to reduce unnecessary disk I/O.

m	bit number
n	key number
k	hash function number
f	False Positive ratio

其中：

$$m \geq n \frac{\log_2(1/\epsilon)}{\ln 2} = n \log_2 e \cdot \log_2(1/\epsilon).$$

k = ln2· (m/n)

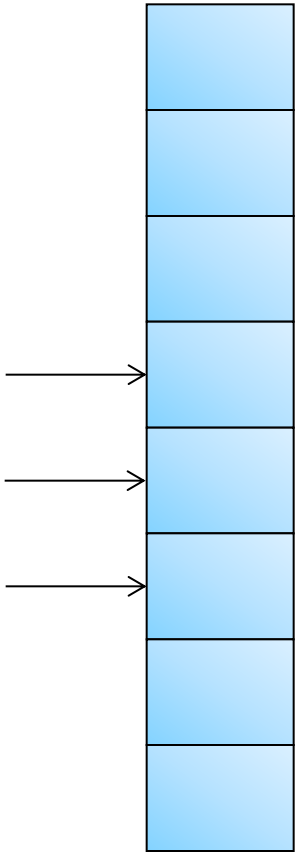
if avage key/value size is 300byte（photo business），then every 4MB key number is13600

e	m	k
0.5	2500KB	1
0.31	4096KB	1
0.1	8192KB	3
0.04	12KB	3

- SSD: 2 KB bloom-filter per MB data; false-positive rate ≈ 0.1
- HDD: 12 KB bloom-filter per 4 MB data; false-positive rate ≈ 0.04

Storage Engine-Disk Layout

If index sub-block > 64 KB: apply binary search with multiple small 4 KB reads instead of one large read to reduce read amplification and latency.



Key size	Sub index block size	read count
16byte	8kb	1*8kb+1*8kb
32byte	8kb	1*8kb+1*8kb
64byte	8kb	1*8kb+1*8kb
128byte	16kb	1*16kb+1*8kb
1KB	128Kb	2*4kb+1*64kb+1*8 kb

Storage Engine—Disk Layout

SSD Perfomance

If index sub-block > 64 KB: apply binary search with multiple small 4 KB reads instead of one large read, to reduce read amplification and latency.

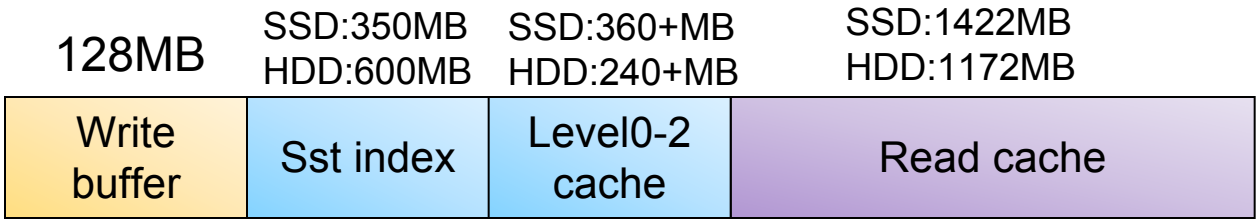
For typical workloads (`key <64 B, value <4 KB`), storage engine can achieve ~10 000 read IOPS on SSD.

read test 1MB SST 8KB sub block										
key size	block size		time cost（us）		time cost（us）		time cost（us）		time cost（us）	iops
1024	128KB			1*128kb	1773	1*8kb	1307	total	3080	2597.402597
1024	128KB	2 *4kb	790	1*64kb	792	1*8kb	510	total	2092	3824.091778
128	16kb			1*64kb	1086	1*8kb	729	total	1815	4407.713499
64	8kb			1*8kb	295	1*8kb	295	total	590	13559.32203
32	8kb			1*8kb	295	1*8kb	295	total	590	13559.32203
16	8kb			1*8kb	295	1*8kb	295	total	590	13559.32203

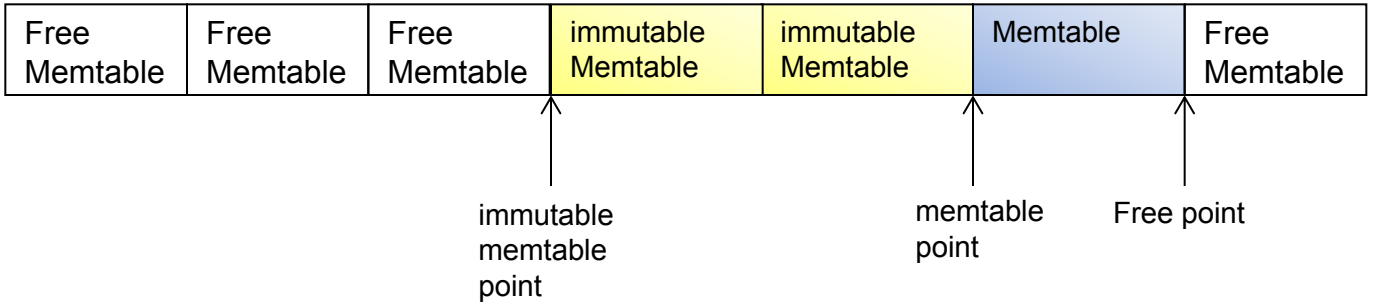
read test 4MB SST 256KB sub block								
key size	block size		time cost（us		time cost（us		time cost（us	iops
1024	16KB	1*16kb	5337	1*256kb	3626	total	8963	111.5698
128	4kb	1*4kb	5274	1*256kb	3721	total	8995	111.1729
64	4kb	1*4kb	5274	1*256kb	3721	total	8995	111.1729
32	4kb	1*4kb	5274	1*256kb	3721	total	8995	111.1729
16	4kb	1*4kb	5274	1*256kb	3721	total	8995	111.1729

Storage Engine— Mem Layout

- Layout Overview

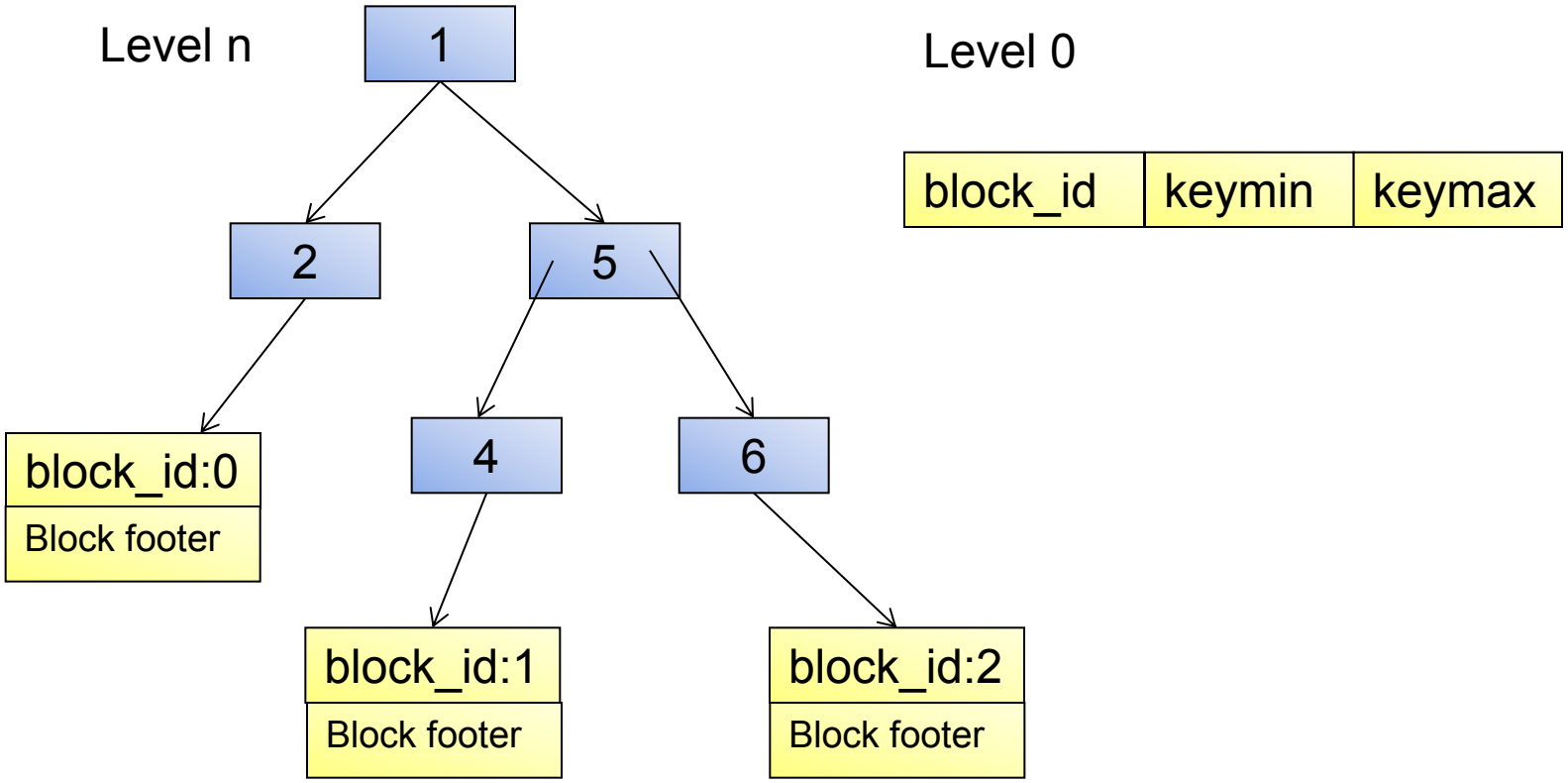


- Write buffer
  - •total size:128MB，every memtable 4MB
  - •Memtable：use skip-list to store kv
  - •Immutable memtable：when memtable is full, it will be sealed and can't write anymore
  - • use circular list to manage memtable & immutable

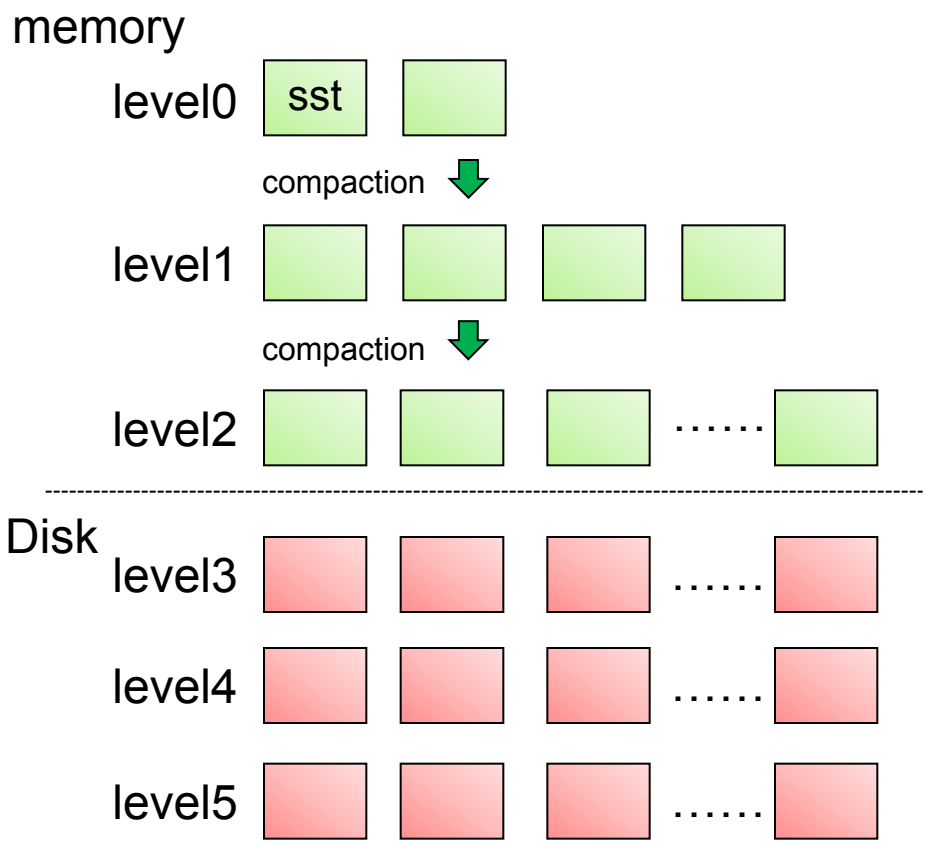


Storage Engine—Memory Layout

- sst index
  - size is 350MB
  - miror of sst index in disk。
  - for each level (except level0), build a 10-way B+-tree indexed by `keymax` over SST blocks.



Storage Engine—Memory Layout



- SST-blocks belonging to level0-level2 are fully cached in memory.
- - Advantages: write amplification reduced by ~10 × compared with original LevelDB; random-write performance improved ~50 %.
- - Risk: power-failure may lose data written within up to ~10 minutes (maximum in-memory data ~600 MB, merge bandwidth 1 MB/s).

Storage Engine—Memory Layout

- Read Cache
  - Cache granularity: SST sub-block.
  - Index key: `(sst-block-id, sub-block-id)`. Implemented via hash table.
  - Eviction policy: LRU. Invalidate cache when underlying block is modified.

Storage Engine—Memory Layout

- Read Cache Allocate Policy

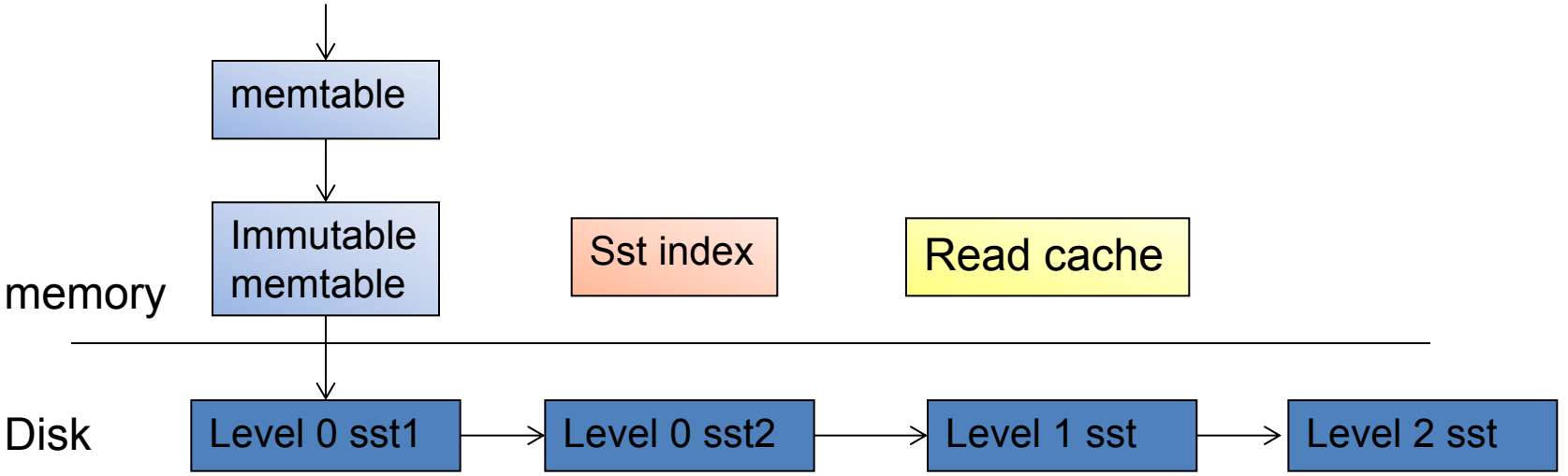
SSD:
HDD:

	Bloom filter	Index sub block(32字节key)
Level 0		
Level 1		
Level 2		
Level 3	6MB	24MB
Level 4	60MB	240MB
Level 5	600MB	2400MB
Level 6	6GB	24GB

Memory budget split across bloom-filter and index sub-blocks for different levels. Remaining memory allocated for data-block read-cache.
With bloom-filter enabled: expected number of physical disk reads for random lookups ≈ 1.2.

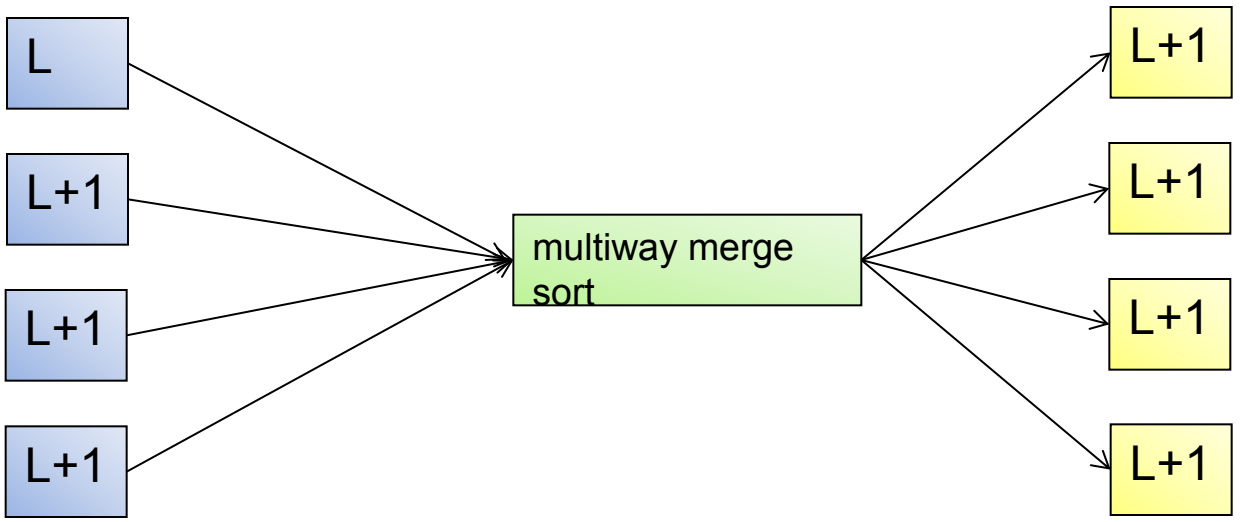
Storage Engine—Read / Write / Delete Workflow

- write/delete: insert record into active memtable and return immediately (write-back style).
- read procedure：



Storage Engine—Compaction Operations

- Memtable-to-level0:
  - when memtable fills up, seal to immutable memtable, convert into SST block and flush to level0.Level n to level n+1
  - Intra-LSM compaction level n → level n+1:
  - - For n > 0: select one source SST block, merge with all overlapping SST blocks in level n+1 using multi-way merge-sort.
  - - For level0: collect all overlapping level0 and level-1 SST files and perform multi-way merge-sort.



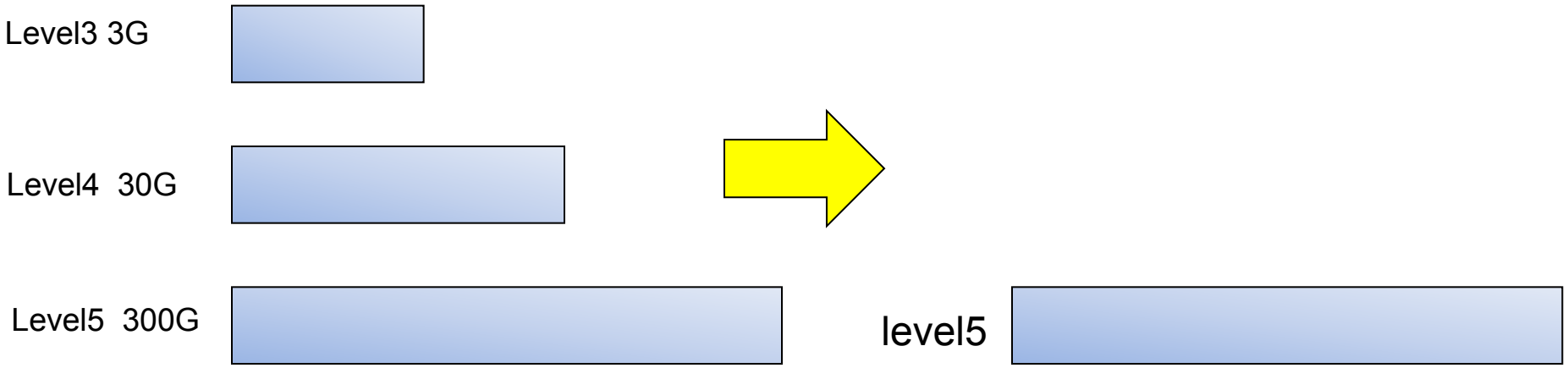
Storage Engine—Compaction Operation

Adaptive compaction:

when disk utilization < 10 % (low traffic), aggressively merge multiple levels into one level to reserve write bandwidth for future traffic spikes.

SSD write-bandwidth observation:

- Before level-3 fills (3 GB): ~40 MB/s write bandwidth
- Before level-4 fills (30 GB): ~6 MB/s write bandwidth
- After level-4 fills: ~3 MB/s write bandwidth



Storage Engine—Performance Comparison

		Leveldb	B+ tree 16K page	Leveldb for SATA	tssdv3
block size		1MB	16K	4MB	\
layers		3	8	4	\
key: 64 Byte value: 200 Byte	index size	400MB	1.2	600MB	1560MB
	random read times	1.2	1	1.16	1
	read amplification	74 (=62*1.2)	62	io amplification is 2*1.16	20
	write amplification	12	62以上	18	40
key: 1024 Byte value: 4096 Byte	index size	400MB	20G (first 7 layer is2G, set cache size 1G	600MB	1560MB
	random read times	1.2	2.5 (1.5 index +1* data read)	1.16	1
	read amplification	5 (=4*1.2)	4	io amplication is 2*1.16	1
	write amplification	12	5 times	18	2

Resource Scheduling

- Resource monitoring
- Shard balancing
- Scheduling policies:
  1. Capacity – triggered scheduling (disk exceeds 85 % threshold)
  2. Access – load – triggered scheduling
  3. Capacity – balancing scheduling
  4. Access – load – balancing scheduling
- Shard operations: split, merge, migrate (migration granularity: per logical shard).

Fault Detection & Recovery

- Fault detection via periodic heartbeat.
- Primary – replica automatic switchover.
- Data – recovery mechanism:
  - Replay unreplicated write – ahead logs.
  - No persistent write – hint.
  - Compare data range by range using sequence – number markers.

Q & A